

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



ACTIVIDAD PRÁCTICA

ACTIVIDAD EN CLASE 03: ESTADO Y RECOMPOSICIÓN

ASIGNATURA

INTRODUCCIÓN AL DESARROLLO DE NUEVAS PLATAFORMAS

TURNO B

DOCENTE

ERNESTO MAURO SUAREZ LOPEZ

INTEGRANTES

JARA MAMANI, MARIEL ALISSON

MESTAS ZEGARRA, CHRISTIAN RAUL

QUISPE CONDORI, ALVARO RAUL

SEQUEIROS CONDORI, LUIS GUSTAVO

AREQUIPA - PERÚ

2026

ACTIVIDAD EN CLASE 03: ESTADO Y RECOMPOSICIÓN

1. Descripción del Aplicativo

El aplicativo implementado corresponde a un sistema de venta de entradas de cine. La solución aborda la recomposición reactiva de la interfaz ante eventos del usuario como la modificación de entradas, adición de productos extras y aplicación de cupones de descuento.

2. Capturas y Evidencias de Implementación

2.1. Entorno de Desarrollo y Vista General

En la Figura 1 se aprecia la vista integral de Android Studio con el código fuente del proyecto y la interfaz de usuario ejecutándose en el emulador móvil.

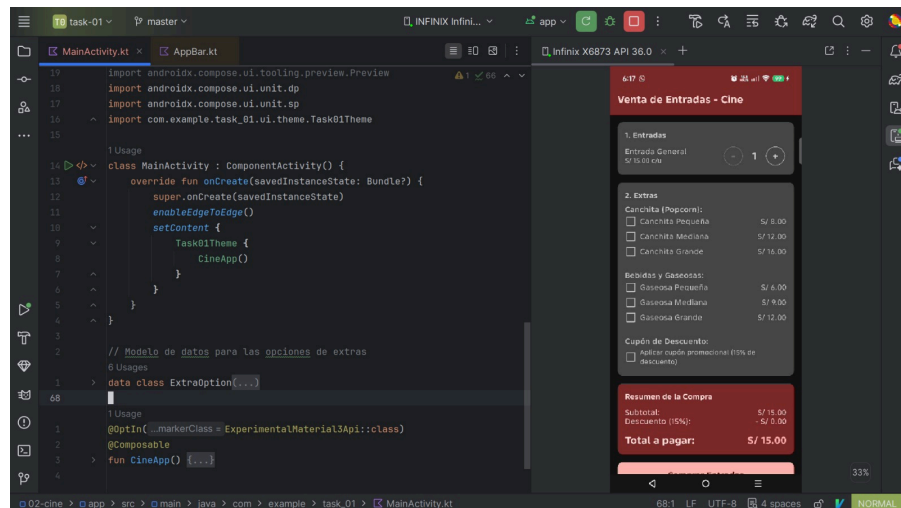


Figura 1. Entorno de desarrollo Android Studio con la interfaz inicial del aplicativo

2.2. Configuración de la Actividad Principal

En la Figura 2 se muestra la clase MainActivity, donde se habilita el diseño de borde a borde y se define el contenedor de contenido con el tema del proyecto y la invocación del composable raíz CineApp().

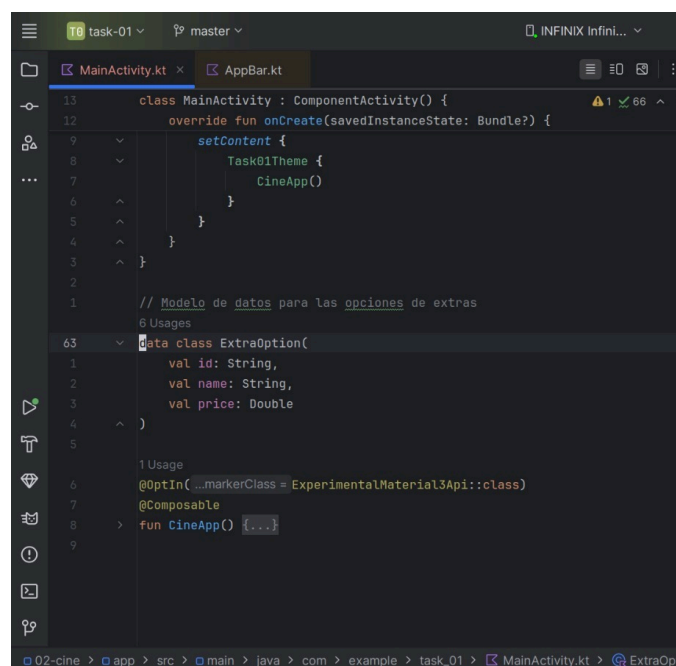


Figura 2. Clase MainActivity y punto de entrada de la composición

2.3. Modelo de Datos para Opciones Adicionales

La Figura 3 ilustra la definición de la clase de datos `ExtraOption`, utilizada para estructurar los elementos adicionales con identificador, nombre y precio unitario.

```
Modelo de datos para las opciones de extras
6 Usages
data class ExtraOption(
    val id: String,
    val name: String,
    val price: Double
)
```

Figura 3. Modelo de datos `ExtraOption` para la gestión de productos

2.4. Catálogo de Opciones y Estado de Selección

En la Figura 4 se observa la definición del catálogo de bebidas y el manejo del estado reactivo de selección, empleando `remember` y `mutableStateMapOf` para conservar las selecciones durante las fases de recomposición.

```
// Opciones de Bebidas / Gaseosas
val bebidaOptions = remember {
    listOf(
        ExtraOption(id = "beb_p", name = "Gaseosa Pequeña", price = 6.00),
        ExtraOption(id = "beb_m", name = "Gaseosa Mediana", price = 9.00),
        ExtraOption(id = "beb_g", name = "Gaseosa Grande", price = 12.00)
    )
}
val selectedBebidas = remember { mutableStateMapOf<String, Boolean>() }
```

Figura 4. Definición de opciones de bebidas y estado mutable mediante `mutableStateMapOf`

2.5. Declaración del Estado de la Interfaz

La Figura 5 expone la inicialización de los estados mutables de la interfaz en `CineApp()`, incluyendo el contador de entradas, las opciones de canchitas y bebidas seleccionadas, y el cupón de descuento.

```
fun CineApp() {
    // --- Estado de la interfaz ---
    var ticketCount by remember { mutableIntStateOf(value = 1) }
    val ticketPrice = 15.00 // Precio por entrada

    // Opciones de Canchita (Popcorn)
    val canchitaOptions = remember {...}
    val selectedCanchitas = remember { mutableStateMapOf<String, Boolean>() }

    // Opciones de Bebidas / Gaseosas
    val bebidaOptions = remember {...}
    val selectedBebidas = remember { mutableStateMapOf<String, Boolean>() }

    // Estado del Cupón de Descuento (Checkbox)
    var applyCoupon by remember { mutableStateOf(value = false) }
    val discountPercentage = if (applyCoupon) 0.15 else 0.0 // 15% de descuento al marcar el checkbox

    // --- Cálculos en memoria ---
    val ticketsSubtotal = ticketCount * ticketPrice

    val canchitasTotal = canchitaOptions.sumOf {...}
    val bebidasTotal = bebidaOptions.sumOf {...}

    val subtotal = ticketsSubtotal + canchitasTotal + bebidasTotal
    val discountAmount = subtotal * discountPercentage
    val total = subtotal - discountAmount

    // Estado para mostrar el cuadro de diálogo de compra
    var showDialog by remember { mutableStateOf(value = false) }

    Scaffold {...} {...}
```

Figura 5. Definición de variables de estado con `remember` y `mutableStateOf`

2.6. Lógica de Cálculos Reactivos en Memoria

En la Figura 6 se detalla la lógica de cálculo en memoria para los subtotales de canchita y bebidas. El sistema calcula dinámicamente el subtotal general, el descuento aplicable y el importe total en respuesta a cualquier cambio de estado.

```
// --- Cálculos en memoria ---  
val ticketsSubtotal = ticketCount * ticketPrice  
  
val canchitasTotal = canchitaOptions.sumOf { option ->  
    if (selectedCanchitas[option.id] == true) option.price else 0.0  
}  
  
val bebidasTotal = bebidaOptions.sumOf { option ->  
    if (selectedBebidas[option.id] == true) option.price else 0.0  
}
```

Figura 6. Cálculo reactivo en memoria de subtotales, descuentos y total a pagar

2.7. Ejecución y Recomposición Dinámica en Móvil

La Figura 7 evidencia el aplicativo en funcionamiento interactivo sobre el dispositivo móvil, demostrando el recálculo automático del resumen de compra tras la selección de elementos adicionales.

6:31 ... 83%

Venta de Entradas - Cine

2. Extras

Canchita (Popcorn):

- ☒ Canchita Pequeña S/ 8.00
- ☐ Canchita Mediana S/ 12.00
- ☐ Canchita Grande S/ 16.00

Bebidas y Gaseosas:

- ☐ Gaseosa Pequeña S/ 6.00
- ☐ Gaseosa Mediana S/ 9.00
- ☐ Gaseosa Grande S/ 12.00

Cupón de Descuento:

- ☐ Aplicar cupón promocional (15% de descuento)

Resumen de la Compra

Subtotal:	S/ 38.00
Descuento (15%):	- S/ 0.00
Total a pagar:	S/ 38.00

Comprar Entradas

Figura 7. Ejecución en dispositivo móvil reflejando la actualización del estado y recomposición